

BERECHENBARKEIT UND KOMPLEXITÄT

Übungsblatt 1

Prof. Rossmanith
C. Grüne, T. Janßen, T. Koglin
Lehrstuhl für Informatik 1
RWTH Aachen

WS 24/25
15.10.2024
Abgabe: Di. 22.10., 16:30

- Die Übungsblätter müssen in Gruppen von 3-4 Studierenden **aus dem gleichen Tutorium** abgegeben werden. Abgaben von Gruppen anderer Größen werden mit 0 Punkten bewertet.
- Die abgegebenen Lösungen müssen mit Namen und Matrikelnummern aller Teammitglieder beschriftet werden.
- Um zur Klausur zugelassen zu werden, muss folgendes erreicht werden: 50% der Punkte in den Blätter 1-6 **und** 50% der Punkte in den Blätter 7-12..
- Es wird nur eine einzelne PDF-Datei als Abgabe akzeptiert. Eingescannte Lösungen sind erlaubt, solange sie gut lesbar sind.
- Die Lösungen zu diesem Blatt werden Di, 22.10., in der Globalübung präsentiert.
- Thema der dieswöchigen Minitests: Formale Definitionen von Sprachen.

Tutoraufgabe 1.1

- Wiederholen Sie die Definitionen der O -, Ω - und Θ -Notation.
- Sortieren Sie folgende Funktionen nach wachsender Größenordnung. Wenn in Ihrer Sortierung f vor g steht, dann ist $f = O(g)$. Begründen Sie dabei jeweils, warum f vor g steht.

$$\sqrt{n}, \quad n^n, \quad \log n, \quad \log(n!), \quad n, \quad n^2, \quad 3^n, \quad n \log n, \quad 2^n$$

Tutoraufgabe 1.2

Geben Sie je eine formale Definition für die Sprachen der folgenden Entscheidungsprobleme an. Machen Sie sich dabei insbesondere Gedanken zur Kodierung der Eingabe und zum Eingabealphabet.

- Eine Unabhängige Menge in einem Graphen $G = (V, E)$ ist eine Menge $I \subseteq V$ von paarweise nicht benachbarten Knoten. Die Sprache L_{UM} enthalte genau die Kodierungen aller Paare (G, ℓ) mit $\ell \in \mathbb{N}$, sodass G eine Unabhängige Menge der Größe mindestens ℓ besitzt.
- Das Teilsummenproblem besteht darin, für eine gegebene Multimenge M von natürlichen Zahlen und eine natürliche Zahl b zu entscheiden, ob es eine Teilmultimenge von M gibt, sodass die Summe der Elemente dieser Teilmultimenge b ist. Die Sprache L_{TS} enthalte genau die Kodierungen aller Paare (M, b) mit dieser Eigenschaft.

Tutoraufgabe 1.3

Entwerfen Sie eine Turingmaschine $M = (Q, \Sigma, \Gamma, B, q_0, \bar{q}, \delta)$ mit $\Sigma = \{0, 1\}$, die für $n \in \mathbb{N}$ schrittweise von n auf 0 herunterzählt, also $n, n-1, \dots, 1, 0$. Die Zahl n werde der Turingmaschine in Binärdarstellung $\text{bin}(n)$ als Eingabe gegeben (das höchstwertige Bit stehe links).

Geben Sie Ihre Turingmaschine formal an und beschreiben Sie kurz ihre Funktionsweise.

Tutoraufgabe 1.4

Geben Sie zu der folgenden Turingmaschine M an, welche Konfigurationen auf der Eingabe $w = 110$ erreicht werden.

$$M = (\{q_0, q_1, \bar{q}\}, \{0, 1\}, \{0, 1, B\}, B, q_0, \bar{q}, \delta)$$

δ	0	1	B
q_0	$(q_0, 0, R)$	$(q_0, 1, R)$	(q_1, B, L)
q_1	$(\bar{q}, 0, R)$	$(q_1, 1, L)$	(q_0, B, R)

Hausaufgabe 1.1

(4 Punkte)

Finden Sie bis zum Ende des 20. Oktober eine Abgabegruppe von 3-4 Studierenden, mit der Sie dieses Blatt abgeben, und registrieren Sie diese auch in Moodle.

Hausaufgabe 1.2

(4 Punkte)

Geben Sie für die Sprachen der folgenden Entscheidungsprobleme jeweils eine formale Definition an. Machen Sie sich dabei insbesondere Gedanken zur Kodierung der Eingabe und zum Eingabealphabet.

- (a) (2 Punkte) Im Verpackungsproblem sind $k \in \mathbb{N}$ Kartons gegeben. Das maximale Füllgewicht eines jeden Kartons ist $W \in \mathbb{N}$. Es liegen $n \in \mathbb{N}$ Gegenstände mit Gewichten $w_1, w_2, \dots, w_n \leq W$ vor. Es ist zu entscheiden, ob die n Gegenstände so auf die k Kartons verteilt werden können, dass für keinen der Kartons das maximale Füllgewicht überschritten wird, oder nicht.

Die Sprache L_{VP} enthalte genau die Kodierungen aller $(n+2)$ -Tupel $(k, W, w_1, \dots, w_n)$ mit dieser Eigenschaft.

- (b) (2 Punkte) Im Problem Kreisfreiheit ist ein endlicher, ungerichteter Graph $G = (V, E)$ gegeben. Es ist zu entscheiden, ob G einen Kreis enthält oder nicht.

Die Sprache L_{KF} enthalte genau die Kodierungen aller ungerichteten Graphen, welche keinen Kreis enthalten.

Hausaufgabe 1.3

(4 Punkte)

Es ist sehr nützlich über einen Turingmaschinensimulator zu verfügen, wenn man Turingmaschinen entwirft. Ziel dieser Aufgabe ist es, einen solchen Simulator zu implementieren.

Der Simulator soll als Eingabe einen Dateinamen einer Datei erhalten, welche eine Beschreibung der zu simulierenden Turingmaschine $M = (Q, \Sigma, \Gamma, B, q_0, \bar{q}, \delta)$ enthält, und ein Eingabewort $w \in \Sigma^*$.

Der Simulator soll dann M auf der Eingabe w simulieren und alle durchlaufenen Konfiguration in jeweils einer Zeile ausgeben (siehe Beispiel).

Die Beschreibung einer Turingmaschine in einer Datei ist dabei folgendermaßen aufgebaut:

1. Die erste Zeile enthält die Anzahl der Zustände n , wobei wir $Q = \{q_1, q_2, \dots, q_n\}$ voraussetzen.
2. Die zweite Zeile enthält alle Zeichen aus Σ .
3. Die dritte Zeile enthält alle Zeichen aus Γ .
4. Die vierte Zeile enthält die Nummer i des Anfangszustands $q_i = q_0$.
5. Die fünfte Zeile enthält die Nummer j des Endzustands $q_j = \bar{q}$.
6. Die folgenden Zeilen enthalten je einen Übergang $\delta: (q, a) \mapsto (q', b, D)$ der Übergangsfunktion δ , wobei q, a, q', b und D in dieser Reihenfolge durch jeweils ein Leerzeichen getrennt auftreten.
7. Wir legen fest, dass das Blanksymbol B ist und nicht umdefiniert wird.

Zum Beispiel eine TM, die das Eingabewort bitweise invertiert, könnte wie folgt aussehen:

```
$ cat invert.TM                                     ...B010[1]10110B...
3                                                     ...B0100[1]0110B...
01                                                    ...B01001[1]110B...
01B                                                  ...B010010[1]10B...
1                                                     ...B0100100[1]0B...
3                                                     ...B01001001[1]BB...
1 0 1 1 R                                           ...B0100100[2]1BB...
1 1 1 0 R                                           ...B010010[2]01BB...
1 B 2 B L                                           ...B01001[2]001BB...
2 0 2 0 L                                           ...B0100[2]1001BB...
2 1 2 1 L                                           ...B010[2]01001BB...
2 B 3 B R                                           ...B01[2]001001BB...
$ java TM.java invert.TM 10110110                  ...B0[2]1001001BB...
...B[1]10110110B...                                ...B[2]01001001BB...
...B0[1]0110110B...                                ...B[2]B01001001BB...
...B01[1]110110B...                                ...BB[3]01001001BB...
```

Die Datei für die TM ist auch im Moodle verfügbar. Testen Sie Ihren Simulator an verschiedenen kleinen Turingmaschinen.

Hinweise zur Implementierung:

- Sie müssen nicht überprüfen, ob Beschreibung und das Eingabewort syntaktisch und semantisch korrekt sind. Bei ungültigen Eingaben darf der Simulator sich beliebig verhalten.
- Es ist erlaubt, die Zeilen für Σ und Γ zu ignorieren und nur die Beschreibung von δ zu verwenden.
- Es ist erlaubt, Übergänge von δ , die nie erreicht werden können, nicht anzugeben.
- Ihr Programm muss nicht hocheffizient sein, sollte aber die TMs in den folgenden Aufgaben schnell simulieren können.
- Verwenden Sie eine halbwegs bekannte Programmiersprache (z.B. Java, C, C++, C#, Python, ...).
- Sie dürfen keine Packages oder Bibliotheken verwenden, die Turingmaschinen simulieren.

Hinweise zur Abgabe:

- Erläutern Sie kurz, wie ihr Simulator funktioniert. Ihr Code sollte auch kurze Kommentare enthalten.
- Geben Sie die Protokolle kleiner Beispielläufe sowie Ihren Quellcode in der .pdf-Datei mit ab.
- Geben Sie außerdem ihren Quelltext in einer entsprechenden separaten Datei ab.

Hausaufgabe 1.4

(4 Punkte)

Entwerfen Sie eine Turingmaschine, welche als Eingabe ein Wort $w \in \{0, 1\}^*$ bekommt, dieses als binär kodierte Zahl interpretiert und die Zahl $\lceil \log_2 w \rceil$ berechnet und ausgibt. Für $w = 0$ und $w = \varepsilon$ soll Ihre TM auch 0 ausgeben. Beschreiben Sie auch die Funktionsweise der TM.

Führen Sie diese Turingmaschine mithilfe des Simulators aus Aufgabe 3 mit den Eingaben 001 und 11010110 aus.

Geben Sie Ihre TM im TM-Format (siehe Aufgabe 3) und die Ausgabe (siehe Aufgabe 3) in der PDF Datei ab. Zusätzlich muss die TM auch als TM-Datei abgegeben werden.

Hausaufgabe 1.5

(4 Punkte)

Entwerfen Sie eine möglichst effiziente (bezogen auf die Anzahl Rechenschritte) Turingmaschine, welche als Eingabe ein Wort $w \in \{0, 1\}^*$ bekommt und entscheidet, ob $2 \cdot |w|_0 = |w|_1$, also die Anzahl an 1en doppelt so groß wie die Anzahl der 0en ist. Beschreiben Sie auch die Funktionsweise der TM.

Führen Sie diese Turingmaschine mithilfe des Simulators aus Aufgabe 3 mit den Eingaben 110100101111 und 110101011001 aus.

Bei syntaktisch nicht korrekten Eingaben kann sich Ihre TM beliebig verhalten.

Geben Sie Ihre TM im TM-Format (siehe Aufgabe 3) und die Ausgabe (siehe Aufgabe 3) in der PDF Datei ab. Zusätzlich muss die TM auch als TM-Datei abgegeben werden.

Aufgabe:	1	2	3	4	5	Total
Punkte:	4	4	4	4	4	20
Erreicht:						

Abgabefrist: Die Lösungen müssen bis **Di. 22.10., 16:30** im Moodle-Lernraum abgegeben werden.